31 fixes

31 copy-paste prompts

App-performance Toolkit.

Every fix you need to make your vibe-coded app *feel instant*, explained simply, with a ready-to-use AI prompt for each one.



App-performance toolkit

Checklist | Prompts

- [x] Skeleton screens
- [x] WebP or AVIF images
- [x] No duplicate API calls
- [x] Debounced inputs
- [ ] + 27 more

Follow **@breakingbreach** for more

© 2026 BreakingBreach · Personal use only

BEFORE YOU START

How to use this *toolkit.*

This toolkit covers all 31 fixes from the video, each with a diagram that shows the idea at a glance. Each one gets a card that explains what it is, why it matters, how to do it, what to watch out for, and how to check it worked, plus a prompt you can paste straight into your AI coding tool.

01

Set up your project

Fill in the short project context on the next page and run the setup prompt. Write “unknown” for anything you're not sure about.

02

Measure and audit

Run the baseline and audit prompts on the Start Here page so you know your starting point and which fixes matter for *your* app.

03

Fix one item at a time

Commit your code, paste one prompt with your context and scope, then review the AI's plan and approve it before anything changes.

04

Verify and repeat

Use each card's verify step, re-measure, and keep going down the list. Finish with the final check prompt.

Anatomy of a card

Number & title
Matches the order in the video.

Category
Where in your app the fix lives.

Impact
How much users will notice it.

Effort
Roughly how much work it takes.

Frontend · Assets · Network · Backend · Database · AI · Infra · Monitoring

How every prompt works

1 Inspect → 2 Propose → 3 Approve → 4 Implement → 5 Verify

- **Small, reversible code changes:** a short plan is enough. If there's no real cost or risk, the AI just says so.
- **Bigger changes** (infrastructure, database migrations, new services, production behaviour): expect the plan to cover cost, downtime, security, compatibility, maintenance and rollback where they apply, with estimates labeled.
- **One approval covers the whole plan.** Deploying to production, enabling paid services, deleting data and production database migrations each need their own clear yes.
- **Every prompt works on its own**, even in a fresh chat. Paste your project context and scope at the top.
- **Keep your coding tool's permission settings on and review the changes.** Prompts guide the AI; they can't enforce anything.

TWO MINUTES, ONCE

Set up your *project.*

Every prompt works better when the AI knows your app. Fill in what you know, write “unknown” for the rest, and paste it at the top of any prompt. Keep the scope small: one page, one endpoint or one service at a time is fine.

PROJECT CONTEXT TEMPLATE

```
PROJECT CONTEXT  (write "unknown" for anything you're not sure about)

App / repo:        [name or link]   Review: [pages, features or folders]
Stack:             [framework] / [hosting] / [database] / [key integrations]
Environment:       [local / development / staging / production]
Key pages & flows: [e.g., home → search → checkout]
Known problems:    [what feels slow]   Measurements: [numbers, or "none yet"]
Scope:             [e.g., the /search page, one endpoint or one service]
May change:        [...]   Must stay the same: [...]

Optional, when relevant:
Traffic:           [rough daily users or requests]
Budget:            [amount]   New paid services OK? [yes / no / ask first]
Constraints:       [downtime limits, data location, sensitive data]
Safety net:        [tests, how you deploy, how you roll back]
```

Then run the setup prompt

SETUP PROMPT

```
Help me set up context for performance work on my app.

Here's what I know so far ("unknown" is fine):
[paste the filled-in project context]

1. Look at the project information you can access (package and config files, schema, deployment files)
and confirm or fill in what you can.
2. Summarize your understanding: stack, environment, key pages and flows, scope, constraints.
3. Ask me up to 5 focused questions, only about gaps that would change your recommendations.
4. Don't guess missing details; mark them "unknown". Don't ask me for passwords, API keys, tokens or
customer data. If something needs them, tell me what to check myself.
5. Don't change anything. End with a short "Project context" block I can paste into the other prompts.
```

> **Never paste secrets**
> Passwords, API keys, tokens and customer data aren't needed to plan performance work. If a check needs them, run it yourself.

PROMPT 00

Start *here.*

Run these two prompts next, with your project context at the top. The first records where you are today. The second checks your app against all 31 items and tells you which ones to do first.

Step 1 · Measure your baseline

BASELINE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Before we optimize anything, help me measure my app's current speed so we can compare later.

1. Pick my 5 most important pages and 5 most-used API endpoints (from my context, or ask me).
2. For each page, run (or tell me how to run) a mobile lab audit with throttling and record LCP, CLS, Total Blocking Time, total JS size, image weight and request count. Lab audits don't measure INP: take it from real-user data if I have any (my monitoring or the Chrome UX Report), otherwise mark it "not measured yet".
3. For each endpoint, measure average and slowest response times and database queries per request.
4. Save the results as a "Before" table in a PERFORMANCE.md file in my repo.

Only measure; don't change app code or settings. If you can't run a measurement, give me the steps and mark it unmeasured. After each fix, we'll add an "After" column.
```

Step 2 · Run the full audit

AUDIT PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Act as a senior performance engineer. Audit my app for speed problems before changing anything.

1. Confirm my stack from the project: framework, hosting, database, ORM and AI provider (if any).
2. Check my app against this 31-point checklist: skeleton screens, image formats, duplicate API calls, debouncing, pagination, CDN, background jobs, N+1 queries, compression, caching, unused dependencies, external API timeouts, AI streaming, optimistic UI, lazy-loading, parallel requests, database indexes, tracing, server/database region, server/static rendering, third-party scripts, code splitting, list virtualization, selected columns, connection pooling, re-renders, browser caching, load balancing, fonts, real-user monitoring, and slow-device testing.
3. Mark each item DONE, NEEDS WORK or NOT RELEVANT, with one line explaining why. Base this on code and measurements you can actually see, and mark anything you couldn't check as UNVERIFIED.
4. Give me a prioritized top-10 fix list, ordered by impact on real users versus effort, and note which fixes involve infrastructure, new services or costs.

Don't change any code yet. Just give me the report.
```

> **Keep the report**
> Save the audit's top-10 list and work through it one item at a time, using the matching card and prompt.

THE CHECKLIST

All 31 fixes *at a glance.*

	#	FIX	CATEGORY	IMPACT	EFFORT
☐	01	Add skeleton loading screens	Frontend	Medium	Low
☐	02	Serve images in WebP or AVIF	Assets	High	Low
☐	03	Prevent duplicate API calls	Network	High	Medium
☐	04	Debounce search and autocomplete inputs	Network	Medium	Low
☐	05	Paginate API results	Backend	High	Medium
☐	06	Serve static files through a CDN	Infra	High	Low
☐	07	Move slow tasks to a background job queue	Backend	High	Medium
☐	08	Fix N+1 queries	Database	High	Medium
☐	09	Enable Brotli or Gzip compression	Network	Medium	Low
☐	10	Cache frequently accessed data with Redis	Backend	High	Medium
☐	11	Remove unused dependencies	Assets	Medium	Low
☐	12	Set timeouts for external API calls	Backend	High	Low
☐	13	Stream AI responses token by token	AI	High	Medium
☐	14	Update the UI before the server responds	Frontend	Medium	Medium
☐	15	Lazy-load images below the screen	Assets	Medium	Low
☐	16	Run independent requests in parallel	Backend	High	Low
☐	17	Add database indexes for common queries	Database	High	Low
☐	18	Trace slow backend requests with OpenTelemetry	Monitoring	High	Medium
☐	19	Place servers close to the database	Infra	High	Low
☐	20	Render pages on the server or at build time	Frontend	High	Medium
☐	21	Load third-party scripts after the page	Frontend	High	Low
☐	22	Split JavaScript by page	Frontend	High	Medium
☐	23	Virtualize long lists	Frontend	Medium	Medium
☐	24	Select only the columns you need	Database	Medium	Low
☐	25	Use database connection pooling	Database	High	Low
☐	26	Prevent unnecessary re-renders	Frontend	Medium	Medium
☐	27	Enable browser caching for static files	Network	Medium	Low
☐	28	Load-balance traffic across multiple app instances	Infra	Medium	Medium
☐	29	Self-host fonts and limit font weights	Assets	Medium	Low
☐	30	Track real-user speed with a performance monitoring tool	Monitoring	High	Low
☐	31	Test on slow phones and networks	Monitoring	High	Low

SHORT ON TIME?

Quick *wins.*

These fixes are **high impact** and **low effort**. If you only have an afternoon, start with these.

- 02 Serve images in WebP or AVIF
- 06 Serve static files through a CDN
- 12 Set timeouts for external API calls
- 16 Run independent requests in parallel
- 17 Add database indexes for common queries
- 19 Place servers close to the database
- 21 Load third-party scripts after the page
- 25 Use database connection pooling
- 30 Track real-user speed with a performance monitoring tool
- 31 Test on slow phones and networks

A note on order

The cards follow the order of the video, not order of importance. The audit prompt on the Start Here page ranks them for your specific app, so use its top-10 list as your real to-do list.



Follow @breakingbreach

for more guides on building fast, secure, and cost-efficient apps.

01 / 31

Add skeleton loading screens.

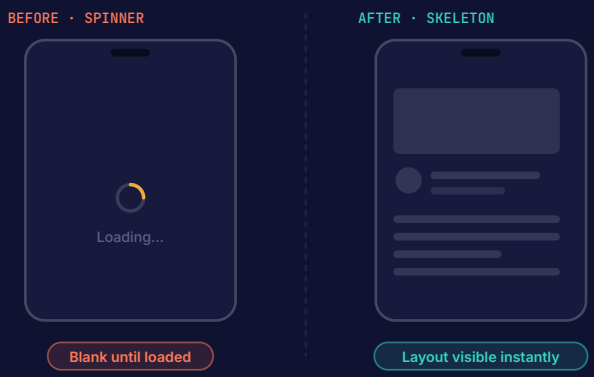
Frontend · IMPACT Medium · EFFORT Low

WHAT IT IS

Show grey placeholder shapes in the exact layout of the content that's about to load, instead of a blank screen or a lone spinner.

WHY IT MATTERS

Users perceive the app as faster because they can see structure immediately, and the page doesn't jump when the real content arrives.



A skeleton shows the page's shape while data loads.

HOW TO DO IT

- Build skeletons that match the final layout's size and position (cards, rows, avatars, text lines).
- Show them only while data is loading, and swap them out the moment it arrives.
- Add `aria-busy="true"` to the loading region so screen readers know it's loading.

WATCH OUT

A skeleton with different dimensions than the real content causes layout shift (CLS). Disable the shimmer animation for users with `prefers-reduced-motion`.

HOW TO VERIFY

Throttle the network in DevTools and reload: you should see the layout instantly, with no jumping when data appears.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Add skeleton loading screens.

1. Find screens and components in scope that show a blank area, a lone spinner or "Loading..."
while fetching data.
2. For each, build a skeleton that matches the final layout (widths, heights, spacing, number of
rows/cards), using one reusable Skeleton component in my existing styling.
3. Add aria-busy="true" to loading regions and turn off the shimmer when prefers-reduced-motion is
set.
4. Make sure nothing shifts when the real content replaces the skeleton.

Keep data-fetching logic unchanged. Report every screen you updated.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

02 / 31

Serve images in WebP or AVIF.

Assets IMPACT High EFFORT Low

WHAT IT IS

Convert JPEG and PNG images to modern formats, and resize them to the size they're actually displayed at.

WHY IT MATTERS

Images are usually the heaviest thing on a page. WebP and AVIF files are typically much smaller than JPEG or PNG at similar quality.



Modern formats and correct sizing cut image weight.

HOW TO DO IT

- Use your framework's image component if it has one; it usually converts, resizes, and lazy-loads for you.
- Otherwise use `<picture>` with an AVIF source, a WebP source, and a JPEG/PNG fallback.
- Serve multiple sizes with `srcset` and `sizes` so phones don't download desktop-sized images.

WATCH OUT

Never ship a 4000px photo into a 400px slot. Always set `width` and `height` (or an aspect ratio) to prevent layout shift.

HOW TO VERIFY

In DevTools → Network → Img, check that images show type avif or webp and that their file sizes dropped.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Optimize images.

1. Find images in scope (<img> tags, CSS backgrounds, images in components) and note format, file size and displayed size.
2. Prefer my framework's image component or my host's image optimization. Otherwise serve AVIF and WebP via <picture> with a JPEG/PNG fallback.
3. Resize to displayed sizes with responsive srcset/sizes, and compress (quality around 75-80 is usually visually lossless; check by eye).
4. Set width and height (or aspect-ratio) on every image to prevent layout shift.
5. Don't lazy-load the main image at the top of the page; give it high fetch priority instead.

Show a before/after table of image weight per page.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

03
/ 31

Prevent duplicate API calls.

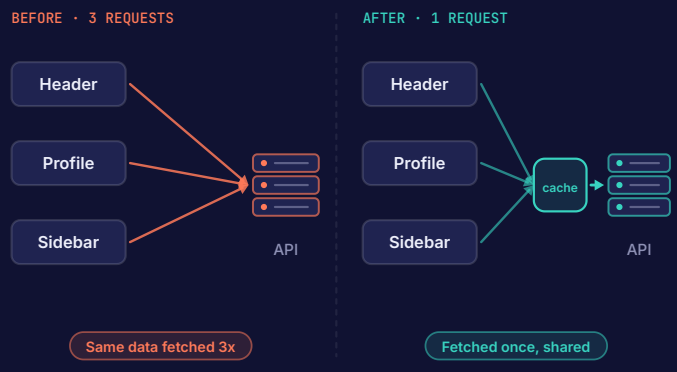
Network IMPACT High EFFORT Medium

WHAT IT IS

Stop the same request from being sent several times at once, for example by multiple components, double-clicks, or re-renders.

WHY IT MATTERS

Duplicate calls waste bandwidth, slow everything down, overload your server, and can even create duplicate records or charges.



Components share one request instead of each fetching.

HOW TO DO IT

- Share one in-flight request between components that need the same data, and cache the result.
- A data-fetching library with built-in caching and deduplication handles most of this automatically.
- Disable submit buttons while a request is pending, and use idempotency keys for create/payment endpoints.

WATCH OUT

In React development mode, StrictMode intentionally runs effects twice. That's normal in dev, but make sure it doesn't happen in production.

HOW TO VERIFY

Open DevTools → Network, load each page, and confirm each endpoint is called once, not several times.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Find and remove duplicate API calls.

1. Load each page in scope and list endpoints called more than once for the same data, with the
cause: several components fetching the same thing, effects re-running, unstable dependencies, or no
caching.
2. Fix it by sharing data through my existing data-fetching layer or state, so each piece is fetched
once and reused. Only add a library if nothing suitable exists, and say why.
3. Disable submit buttons while requests are pending so double-clicks can't send twice.
4. For create or payment endpoints, propose idempotency keys. This changes the API, so explain the
client and server changes.

Ignore React StrictMode's double-run in development. List each duplicate and how you fixed it.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

04
/ 31

Debounce search and autocomplete inputs.

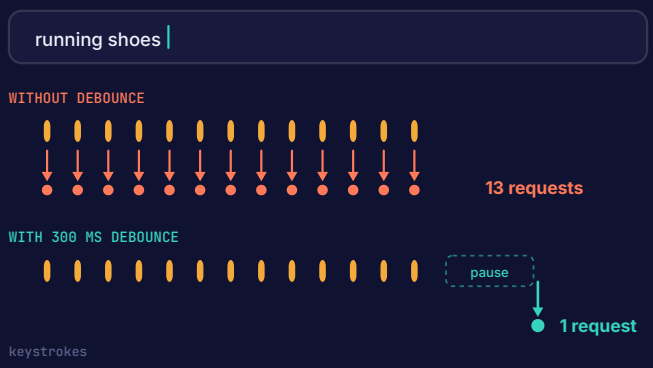
Network IMPACT Medium EFFORT Low

WHAT IT IS

Wait until the user pauses typing (around 250–400 ms) before sending a search request, instead of firing one on every keystroke.

WHY IT MATTERS

Typing "running shoes" without debouncing sends 13 requests. With debouncing it sends one or two.



One request after the user pauses, not one per key.

HOW TO DO IT

- Wrap the search call in a debounce of about 300 ms.
- Cancel the previous request when a new one starts (for example with `AbortController`).
- Skip searches for very short queries (for example, fewer than 2 characters).

WATCH OUT

Without cancelling, an older slow response can arrive after a newer one and overwrite the correct results.

HOW TO VERIFY

Type quickly in the search box with the Network tab open: you should see one request after you stop typing.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Debounce search, filter and autocomplete inputs.

1. Find inputs in scope that trigger an API call or expensive filtering on every keystroke.
2. Add a debounce of about 300 ms through one reusable hook or utility.
3. Cancel the previous in-flight request when a new one starts (AbortController), so stale
results never overwrite newer ones.
4. Wait for at least 2 characters and show a small loading indicator while searching.
5. Keep typing instant; only the request should be delayed.

Report each input you changed and the request count before/after for a sample query.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

05 / 31

Paginate API results.

Backend · IMPACT High · EFFORT Medium

WHAT IT IS

Return data in pages (for example 20–50 items at a time) instead of sending every row in one response.

WHY IT MATTERS

Endpoints that return everything get slower as your data grows, and eventually time out or crash the browser.



Send a page of results, not the whole table.

HOW TO DO IT

- Add a limit and a cursor (or page) parameter to every list endpoint.
- Prefer cursor-based (keyset) pagination for large or fast-changing tables; it stays fast where offset pagination slows down.
- Enforce a maximum page size on the server, so nobody can request a million rows.

WATCH OUT

Offset pagination (`OFFSET 100000`) gets slower the deeper you go, and can skip or repeat rows when data changes between pages.

HOW TO VERIFY

Call each list endpoint and confirm it returns a limited number of items plus a next-page cursor.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Add pagination to list endpoints.

1. Find routes and queries in scope that return lists without a limit, and note how many rows they
return today.
2. Use cursor-based pagination (limit + cursor, ordered by a unique, indexed column) that returns a
nextCursor. Default to 20 items and enforce a maximum (e.g., 100) on the server.
3. Make sure the ordering column is indexed.
4. Update the frontend with "Load more", infinite scroll or page buttons, whichever fits each
screen.

This changes API responses: list affected consumers and, where needed, propose a backward-compatible
option (e.g., new parameters with the old behaviour as the default).

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

06
/ 31

Serve static files through a CDN.

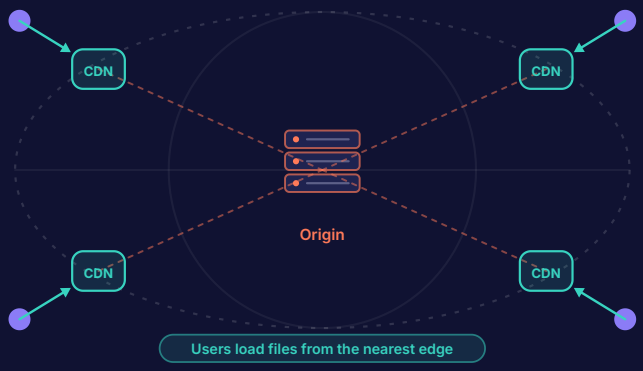
Infra IMPACT High EFFORT Low

WHAT IT IS

Deliver images, JavaScript, CSS, and fonts from a content delivery network that caches copies on servers around the world.

WHY IT MATTERS

Files load from a location near each user instead of travelling all the way to your origin server.



Static files are served from an edge near each user.

HOW TO DO IT

- Check your hosting platform first; many already put static files behind a CDN.
- Otherwise put a CDN in front of your static assets or storage bucket.
- Use versioned (hashed) file names so you can cache them for a long time.

WATCH OUT

Never let the CDN cache personalized or logged-in responses, or one user may see another user's page.

HOW TO VERIFY

Check the response headers of a static file for a cache status such as HIT, and compare load times from different regions.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Make sure static assets are served through a CDN.

1. Check how my app is hosted and whether static files (JS, CSS, images, fonts, videos) already go
through a CDN. If they do, say so and report only gaps.
2. Prefer my host's built-in CDN before adding a new provider.
3. Make sure asset file names are versioned (content hashes) so they can be cached long-term.
4. Confirm that HTML, API responses and anything personalized or behind login are NOT cached by the
CDN.
5. Explain how to check cache hits in the response headers.

Because this touches infrastructure, cover: setup and ongoing costs, DNS or config changes, and
rollback. Label estimates and note what to verify (e.g., provider pricing). List any dashboard steps
I need to do myself.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

07 / 31

Move slow tasks to a background job queue.

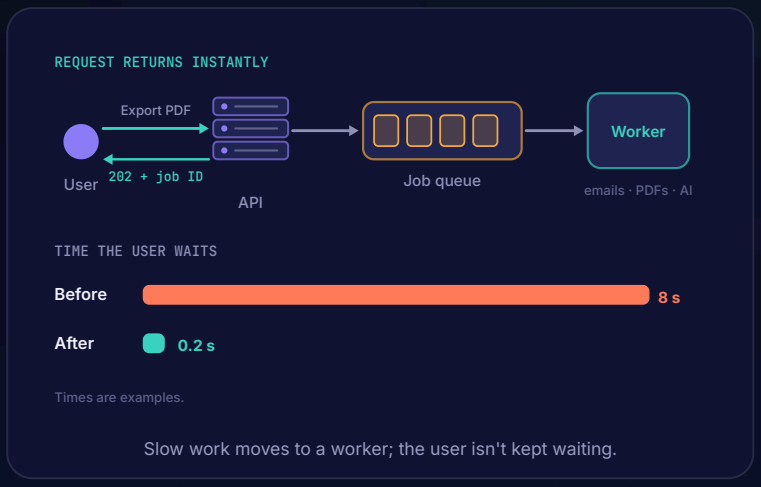
Backend · IMPACT High · EFFORT Medium

WHAT IT IS

Run slow work (sending emails, generating PDFs, processing videos, bulk AI calls) in a background worker instead of making the user wait for it.

WHY IT MATTERS

The request returns instantly, and heavy work no longer blocks your server from handling other users.



Slow work moves to a worker; the user isn't kept waiting.

HOW TO DO IT

- Push the task onto a job queue and immediately return a job ID to the user.
- Process jobs in a separate worker with retries, backoff, and a timeout.
- Show progress or notify the user when the job is done.

WATCH OUT

Make jobs safe to run twice (idempotent), because retries happen. On serverless platforms, work started after the response is sent may be killed, so use a real queue.

HOW TO VERIFY

Trigger the slow action: the request should return in well under a second, and the result should appear when the job finishes.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Move slow work out of request handlers into a background job queue.

1. Find routes in scope that take more than ~1 second because of emails, file generation, image/video
processing, webhooks or bulk AI calls. Measure or estimate their current time (say which).
2. Check what my stack and host already provide (built-in queues, background or scheduled functions)
before suggesting a new service.
3. Change those routes to enqueue a job and immediately return a job ID (HTTP 202). Process jobs in a
worker with retries, exponential backoff, a max attempt count and a timeout.
4. Make each job idempotent so a retry never sends two emails or charges twice.
5. Add a way for the frontend to check job status or get notified.

Because this touches new infrastructure, cover: where the worker runs, setup and usage-based costs,
monitoring, and rollback. Label estimates and note what to verify (e.g., provider pricing). Report old
vs. new response times.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about
gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs.
Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan
unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for
anything you couldn't run. Production deploys, new paid services, deleting data and production migrations
each need my separate yes.
```

08
/ 31

Fix N+1 queries.

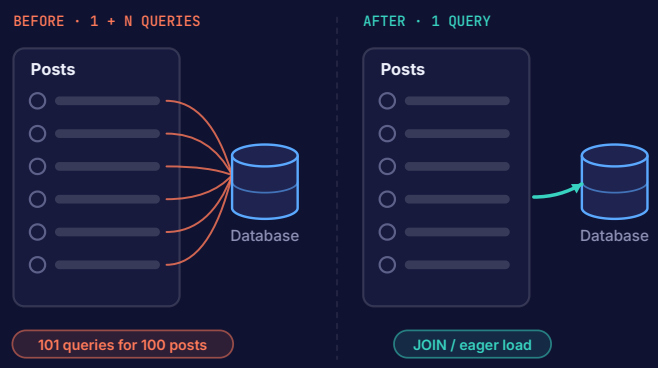
Database IMPACT High EFFORT Medium

WHAT IT IS

An N+1 problem is when your code runs one query to get a list, then one extra query for every item in that list.

WHY IT MATTERS

Loading 100 posts with their authors can turn into 101 database queries instead of 1 or 2, and it's one of the most common slowdowns in generated code.



Fetch related data in one query instead of one per row.

HOW TO DO IT

- Turn on query logging and count queries per request.
- Fetch related data in one go with a JOIN or your ORM's include/eager-loading option.
- In GraphQL or nested resolvers, batch lookups so each type is loaded once per request.

WATCH OUT

Eager-loading everything can over-fetch. Only include the relations the page actually uses.

HOW TO VERIFY

With query logging on, load the page: the number of queries should stay the same whether there are 10 items or 1,000.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Find and fix N+1 queries.

1. Enable query logging (in development or staging) and find requests in scope where the number
of queries grows with the number of items returned.
2. Look especially for database calls inside loops, .map() calls, nested components or GraphQL
resolvers.
3. Fix each one with a single query (JOIN or my ORM's include/eager loading) or batched lookups,
loading only the relations the page uses.
4. Keep the response data exactly the same.

For each fix, show the query count before and after.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

09 / 31

Enable Brotli or Gzip compression.

Network · IMPACT Medium · EFFORT Low

WHAT IT IS

Compress text responses (HTML, CSS, JavaScript, JSON, SVG) before sending them, so less data travels over the network.

WHY IT MATTERS

Text files often shrink by 70% or more, which is a big win on slow mobile connections.



Text is compressed on the way out; media is skipped.

HOW TO DO IT

- Check your host or CDN first; many enable compression by default.
- Prefer Brotli and fall back to Gzip for older clients.
- For static files, pre-compress them at build time so the server doesn't compress on every request.

WATCH OUT

Don't compress files that are already compressed, like JPEG, PNG, WebP, AVIF, MP4, or ZIP. It wastes CPU and saves nothing.

HOW TO VERIFY

In DevTools → Network, check that text responses have a `content-encoding` header of br or gzip.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Check that text responses are compressed.

1. Check whether my host, CDN or server already applies Brotli or Gzip. "Already configured" is
a fine answer.
2. If not, enable it for HTML, CSS, JS, JSON and SVG (Brotli preferred, Gzip fallback), and pre-
compress static assets at build time if my setup supports it.
3. Exclude already-compressed formats (images, video, archives) and tiny responses.
4. Make sure streaming responses (like AI output) still stream and aren't buffered by
compression.

Show me how to confirm it with the content-encoding header.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

10 / 31

Cache frequently accessed data with Redis.

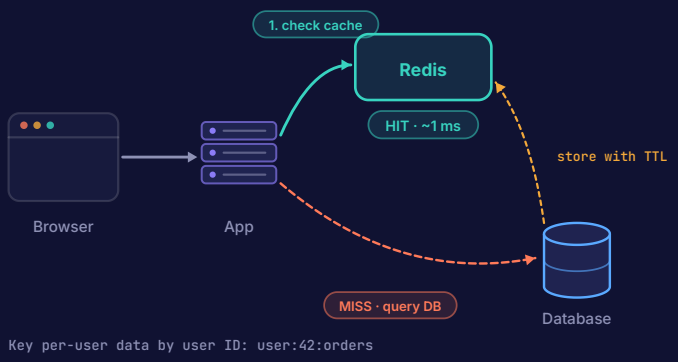
Backend · IMPACT High · EFFORT Medium

WHAT IT IS

Keep the results of frequent, expensive queries in Redis (or a similar in-memory store), so most requests skip the database entirely.

WHY IT MATTERS

Reading from memory takes about a millisecond, while a slow database query can take hundreds.



Most reads hit Redis; only misses reach the database.

HOW TO DO IT

- Use the cache-aside pattern: check the cache, fall back to the database on a miss, then store the result.
- Give every cache entry an expiry time (TTL), and delete or update it when the underlying data changes.
- Start with data that's read often and changes rarely: settings, product lists, public profiles, counts.

WATCH OUT

If a cache key doesn't include the user or account ID, one user can see another user's data. Never cache user-specific data under a shared key.

HOW TO VERIFY

Log cache hits and misses: repeat requests should be hits, and response times should drop.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Add Redis caching for frequently accessed data.

1. Find the most frequent and slowest reads in scope (use logs or tracing if available). Pick data
that's read often and changes rarely.
2. Check whether Redis or another cache already exists and prefer it.
3. Implement cache-aside with a small helper: get from cache → on miss, query the DB → store with a
TTL. Invalidate or update the entry whenever that data is written.
4. For user- or account-specific data, include the user/account ID in the cache key so no one can
see another user's data. Don't cache secrets.
5. If the cache is unavailable, fall back to the database instead of crashing.

If this adds a Redis service, cover its cost, region and what data it will hold. Report what you
cached, the key format, the TTL, and measured hit rate or timings.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

11 / 31

Remove unused dependencies.

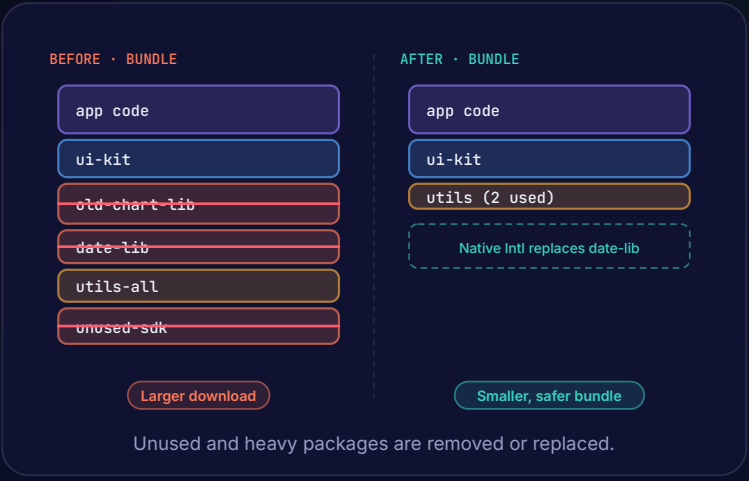
Assets · IMPACT Medium · EFFORT Low

WHAT IT IS

Delete packages you no longer use, and replace heavy libraries with lighter alternatives or built-in browser features.

WHY IT MATTERS

AI tools often install packages and never remove them. Unused code bloats your bundle, slows installs and builds, and adds security risk.



Unused and heavy packages are removed or replaced.

HOW TO DO IT

- Run a dependency-checking tool to list unused packages, and a bundle analyzer to find the heaviest ones.
- Replace heavy libraries where native features work (for example, date formatting with `Intl`).
- Import only what you use instead of whole libraries.

WATCH OUT

Some packages are used only in config files, scripts, or at build time. Double-check before deleting, and run a full build and tests afterwards.

HOW TO VERIFY

Compare bundle size and install size before and after, and make sure the app builds and runs normally.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Clean up my project's dependencies.

1. Check each package in my package file for real usage, including config files, scripts and build steps.
2. Propose removing the ones that are truly unused.
3. Analyze the production bundle and list the 10 heaviest dependencies. For each, suggest a lighter alternative or a native browser/runtime feature, and swap it only where behaviour stays the same.
4. Change whole-library imports to specific imports where that reduces bundle size.
5. Run the build and tests to confirm nothing broke.

Show bundle size before and after.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

12 / 31

Set timeouts for external API calls.

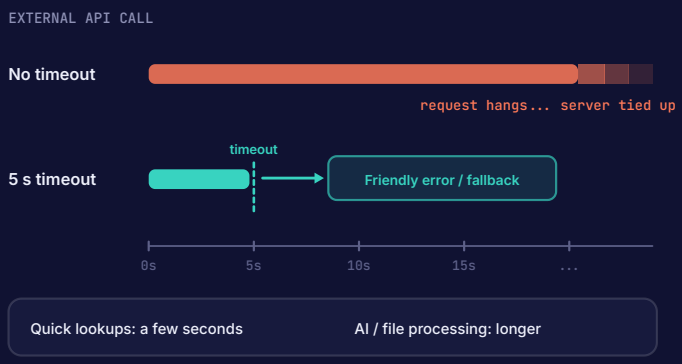
Backend IMPACT High EFFORT Low

WHAT IT IS

Give every call to a third-party service (payments, email, AI, maps) a maximum time to wait before giving up.

WHY IT MATTERS

Without a timeout, one slow external service can hang your requests and tie up your server, making the whole app feel broken.



A timeout turns a hung request into a quick fallback.

HOW TO DO IT

- Set a timeout on every outbound request (for example with `AbortSignal.timeout()`).
- Use short timeouts for quick lookups and longer ones for AI or file processing.
- Show a clear error or fallback when a call times out, and retry only when it's safe.

WATCH OUT

Many HTTP clients wait much longer than users will by default. Retrying non-idempotent calls (like payments) after a timeout can cause duplicates unless you use idempotency keys.

HOW TO VERIFY

Point a call at a deliberately slow endpoint: your app should give up at the timeout and show a friendly error.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Add timeouts to external API calls.

1. Find outbound HTTP requests and third-party SDK calls in scope (payments, email, AI, storage, maps, webhooks) and note any existing timeouts.
2. Give each an explicit timeout: a few seconds for quick lookups, longer for AI generation or file processing. Route them through one shared helper.
3. On timeout, return a clear error or fallback instead of hanging.
4. Retry only safe, idempotent calls, with backoff and a max attempt count. Never blindly retry payments; use the provider's idempotency keys if available.

List each call with the timeout you chose and why.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

13 / 31

Stream AI responses token by token.

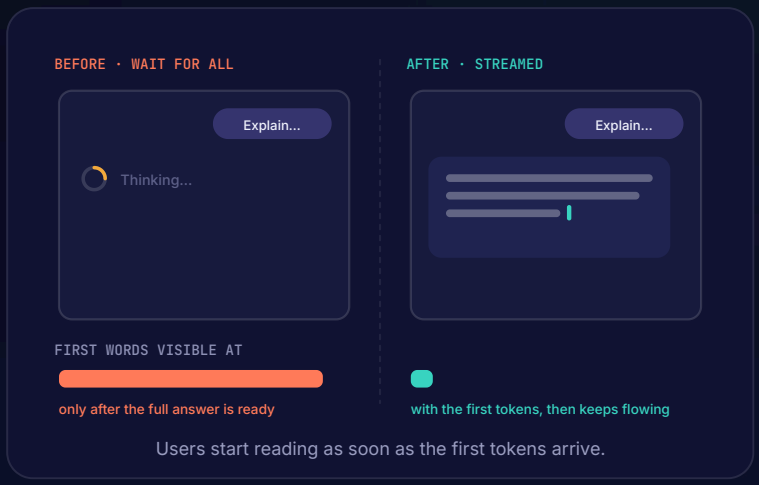
AI · IMPACT High · EFFORT Medium

WHAT IT IS

Show AI output word by word as it's generated, instead of waiting for the full answer and showing it all at once.

WHY IT MATTERS

Users start reading as soon as the first tokens arrive instead of waiting for the whole answer, so the app feels much faster even though the total time is similar.



HOW TO DO IT

- Enable streaming in your AI provider's API call.
- Send the stream to the browser (for example with server-sent events) and render text as it arrives.
- Stop generation when the user cancels or leaves the page.

WATCH OUT

Some proxies, compression settings, or hosting setups buffer responses, which breaks streaming. Check that your host supports streaming responses.

HOW TO VERIFY

Measure time to first token before and after, and check in the Network tab that text arrives in many small chunks rather than all at once.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Stream AI responses token by token.

1. Find places in scope where my app waits for a full AI response, and measure the time to first
visible text today.
2. Enable streaming in the provider call and forward it to the frontend (server-sent events or a
streamed response).
3. Render text as it arrives, including markdown, without the layout jumping. Add a Stop button that
aborts the upstream request.
4. Check that nothing buffers the stream (proxies, compression, hosting settings), and handle errors
mid-stream.
5. Keep my prompts and model settings unchanged.

Report time to first token before and after, and confirm in the Network tab that text arrives
progressively rather than in one chunk.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about
gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs.
Wait for my approval before editing code, config, infrastructure or data; approval covers the whole
plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual
steps for anything you couldn't run. Production deploys, new paid services, deleting data and
production migrations each need my separate yes.
```

14 / 31

Update the UI before the server responds.

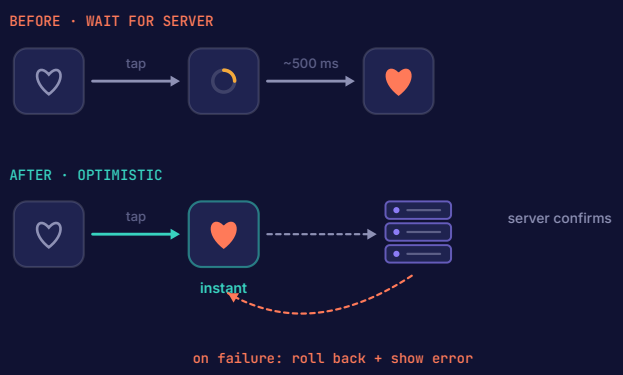
Frontend · IMPACT Medium · EFFORT Medium

WHAT IT IS

Called an optimistic update: show the result of an action (like, add to cart, rename) instantly, then confirm it with the server in the background.

WHY IT MATTERS

Actions feel instant instead of waiting for a round trip to the server.



The UI updates first; the server confirms after.

HOW TO DO IT

- Update local state right away when the user acts.
- Send the request in the background.
- If it fails, roll back to the previous state and show an error.

WATCH OUT

This is a display trick only. The server must still validate everything. Don't use it for payments or actions where showing a false success would be harmful.

HOW TO VERIFY

Throttle the network and click the action: the UI should update immediately and roll back cleanly if you force a failure.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Add optimistic UI updates to quick actions.

1. Find small, low-risk actions in scope where users wait for the server: likes, toggles,
adding/removing items, renaming, reordering, marking as done.
2. For each, update the UI immediately, send the request in the background, and roll back with
an error message if it fails.
3. Keep the server as the source of truth and reconcile with its response.
4. Don't apply this to payments, irreversible deletions or security-sensitive actions.

List each action you made optimistic.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

15 / 31

Lazy-load images below the screen.

Assets · IMPACT Medium · EFFORT Low

WHAT IT IS

Only load images and videos when the user scrolls near them, instead of downloading everything when the page opens.

WHY IT MATTERS

The first screen loads faster because the browser isn't busy downloading images nobody has seen yet.



Only what's on screen loads up front.

HOW TO DO IT

- Add `loading="lazy"` to images and iframes that start below the first screen.
- For videos, use `preload="none"` (or `metadata`) with a poster image.
- Always set width and height so the page doesn't jump when images load.

WATCH OUT

Never lazy-load the main image at the top of the page (usually the Largest Contentful Paint element). It makes it appear later. Give it high fetch priority instead.

HOW TO VERIFY

Open the Network tab and reload: below-the-fold images should only start downloading as you scroll.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Lazy-load images, videos and embeds below the first screen.

1. Find images, videos and iframes (embeds, maps, players) in scope.
2. Add loading="lazy" to images and iframes that start below the first screen. For videos, use
preload="none" or "metadata" with a poster image.
3. Don't lazy-load the main image at the top of each page; give it fetchpriority="high" instead.
4. Make sure every image has width/height or an aspect ratio to prevent layout shift.

List what you lazy-loaded and what you prioritized.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

16 / 31

Run independent requests in parallel.

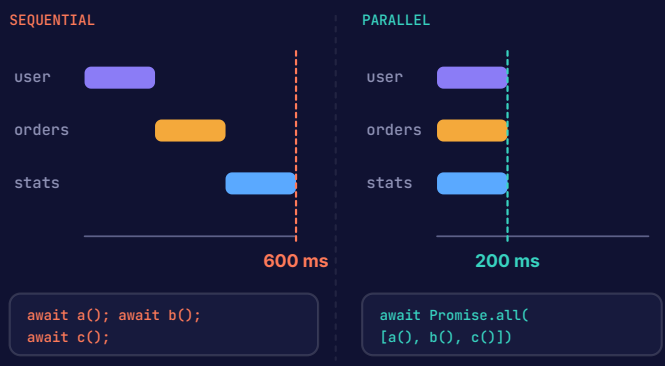
Backend IMPACT High EFFORT Low

WHAT IT IS

When several requests or queries don't depend on each other, start them all at once instead of waiting for each to finish before starting the next.

WHY IT MATTERS

Three 200 ms calls take about 600 ms one after another, but about 200 ms in parallel.



Independent calls run together, not one after another.

HOW TO DO IT

- Replace chains of `await` with `Promise.all` (or `Promise.allSettled` if some may fail).
- On the frontend, start fetches at the page level instead of nesting them in child components (request waterfalls).
- Cap concurrency when there are many calls, so you don't overload a service.

WATCH OUT

Only parallelize work that is truly independent. If request B needs the result of request A, it must wait.

HOW TO VERIFY

Look at the Network waterfall or your traces: independent requests should start at the same time.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Run independent requests in parallel.

1. Find independent awaits running one after another in scope (API calls, database queries, file reads),
in frontend and backend.
2. Run truly independent ones with Promise.all, or Promise.allSettled when partial failure is acceptable.
3. Fix frontend waterfalls (a child only fetching after its parent finishes) by starting those fetches
earlier or together.
4. Limit concurrency for large loops so no service is overloaded.
5. Leave calls that depend on each other sequential.

Show each change with the time saved, and say whether it was measured or estimated.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about
gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs.
Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan
unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for
anything you couldn't run. Production deploys, new paid services, deleting data and production migrations
each need my separate yes.
```

Halfway there. Enjoying the toolkit?

Follow @breakingbreach for more guides on building fast, secure, and cost-efficient apps.

17 / 31

Add database indexes for common queries.

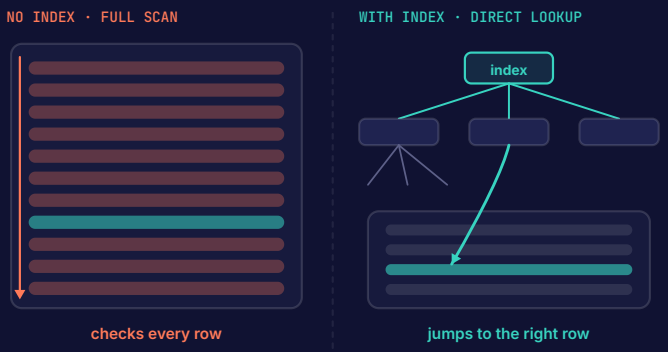
Database · IMPACT High · EFFORT Low

WHAT IT IS

An index lets the database find matching rows directly, instead of scanning the entire table for every query.

WHY IT MATTERS

A missing index can make a query hundreds of times slower once a table grows to thousands or millions of rows.



An index finds rows directly instead of scanning.

HOW TO DO IT

- Index columns used in `WHERE`, `JOIN`, and `ORDER BY`, especially foreign keys.
- Use composite indexes for queries that filter on several columns together (column order matters).
- Check slow queries with `EXPLAIN` to see which plan the database chooses.

WATCH OUT

Every index slows down writes and uses storage, so don't index everything. On large production tables, use a lower-locking method such as `CREATE INDEX CONCURRENTLY` in Postgres (slower, and it can fail partway).

HOW TO VERIFY

Compare `EXPLAIN ANALYZE` plans and timings before and after. A sequential scan can still be the right plan for small tables or broad queries.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Add the right database indexes.

1. Read my schema and the queries in scope (including ORM queries), and find slow or frequent ones from logs or EXPLAIN.
2. Find columns used in WHERE, JOIN and ORDER BY without a suitable index, especially foreign keys and lookup columns (email, slug, user_id, created_at).
3. Propose indexes, including composite indexes in the right column order, with the reason for each. Skip duplicates and nice-to-haves: every index slows writes and uses storage.
4. Write them as a migration. On large production tables, use a lower-locking method (e.g., CREATE INDEX CONCURRENTLY in Postgres: slower, can't run in a transaction, can fail partway).

Compare EXPLAIN ANALYZE plans and timings before and after; a sequential scan can still be right for small tables or broad queries.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

18 / 31

Trace slow backend requests with OpenTelemetry.

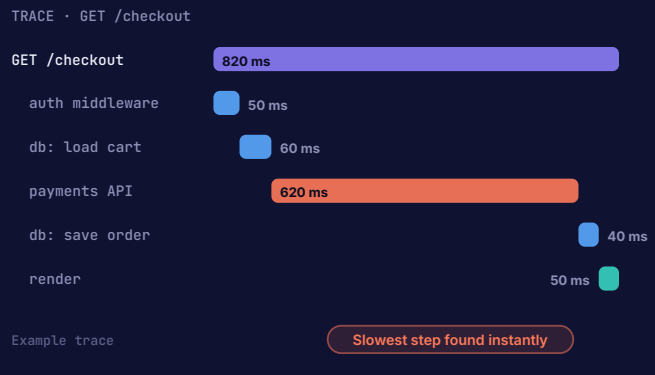
Monitoring IMPACT High EFFORT Medium

WHAT IT IS

Tracing records how long each step of a request takes (database queries, external calls, your own code), so you can see exactly where time goes. OpenTelemetry is an open standard for collecting traces.

WHY IT MATTERS

Instead of guessing why an endpoint is slow, you see a timeline showing the exact query or call that's responsible.



A trace shows exactly which step is slow.

HOW TO DO IT

- Add OpenTelemetry instrumentation for your HTTP server, database client, and outbound HTTP calls.
- Send traces to a tracing backend of your choice.
- Add custom spans around important business logic.

WATCH OUT

Sample traces in production to control volume and cost, and never put passwords, tokens, or personal data in trace attributes.

HOW TO VERIFY

Make a few requests and open your tracing backend: you should see each request broken into timed steps.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Add OpenTelemetry tracing to my backend.

1. Check what monitoring or tracing already exists; my host may already provide some.
2. Set up the OpenTelemetry SDK for my language and framework, with automatic instrumentation for
incoming HTTP requests, database queries and outbound HTTP calls.
3. Add custom spans around important logic in scope (e.g., checkout, AI calls, file processing).
4. Configure the exporter from environment variables and use sampling in production.
5. Keep secrets, tokens, passwords and personal data out of span attributes and logs.

Because this touches a new tracing service, cover: setup and usage-based costs, data retention, and
where trace data is stored. Label estimates and note what to verify (e.g., provider pricing). Then
show me how to find my 5 slowest endpoints.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

19 / 31

Place servers close to the database.

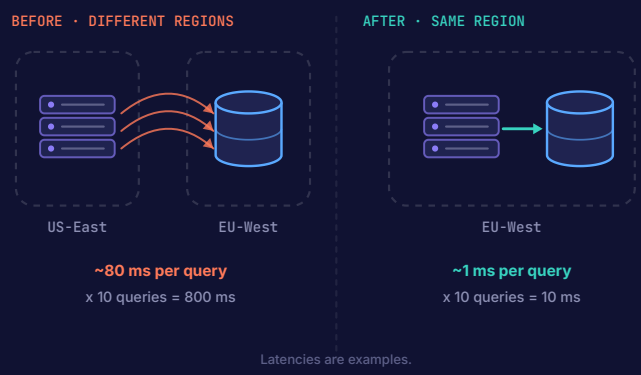
Infra IMPACT High EFFORT Low

WHAT IT IS

Run your app server (or serverless functions) in the same region as your database.

WHY IT MATTERS

Every query is a round trip. If your server is in one region and your database in another, a page with 10 queries can add hundreds of milliseconds.



Every query pays the distance between server and database.

HOW TO DO IT

- Check which region your database is in and which region your server or functions run in.
- Move one of them so they match.
- If users are worldwide, use a CDN for static content and read replicas for reads in other regions.

WATCH OUT

Running database-heavy code at the edge, far from the database, often makes things slower, not faster.

HOW TO VERIFY

Compare average query time or total server response time before and after moving.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Check whether my servers and database are in the same region.

1. Read my hosting, serverless and database config and list the region each part runs in. If they
already match, say so.
2. If they differ, measure or estimate the added latency per request (say which) and outline how to
align them.
3. Find edge functions or middleware that make database calls, and suggest running those in the
database's region.
4. If my users are global, say whether read replicas or caching would help.

Because this touches infrastructure, cover: downtime, data migration, any data-location
requirements, cost changes, and rollback. Label estimates and note what to verify (e.g., provider
pricing). Give exact dashboard steps I need to do myself.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

20 / 31

Render pages on the server or at build time.

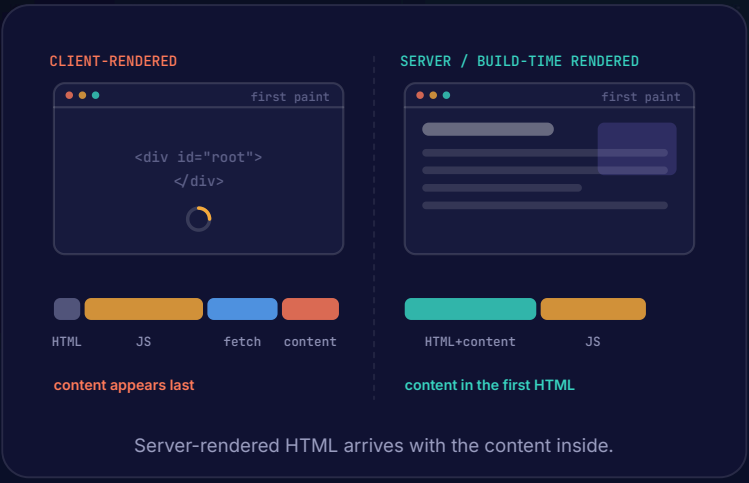
Frontend · IMPACT High · EFFORT Medium

WHAT IT IS

Send ready-made HTML instead of an empty page that the browser has to build with JavaScript. Pages that rarely change can be generated once at build time.

WHY IT MATTERS

Users see content much sooner, and search engines and link previews can read your pages.



Server-rendered HTML arrives with the content inside.

HOW TO DO IT

- Generate static pages (blog, docs, marketing pages) at build time.
- Render dynamic but public pages on the server.
- Keep client-side rendering for highly interactive, logged-in screens where SEO doesn't matter.

WATCH OUT

Be careful not to include secrets or other users' private data in server-rendered HTML or the page's embedded data.

HOW TO VERIFY

Use View Page Source: your main content should be in the HTML, not just an empty root element.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Improve how my pages are rendered.

1. List the routes in scope and how each is rendered now (client, server or static).
2. Recommend a mode for each: static for content that rarely changes, server rendering for public
dynamic pages, client rendering for interactive logged-in screens. "Keep as is" is fine.
3. Implement with my framework's built-in features, adding revalidation for static pages that
change occasionally.
4. Keep secrets and private user data out of the rendered HTML and embedded page data.
5. Add proper titles and meta tags while you're there.

Note any hosting cost or behaviour changes (e.g., more server work, caching). Show the page list
with the mode chosen for each.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

21 / 31

Load third-party scripts after the page.

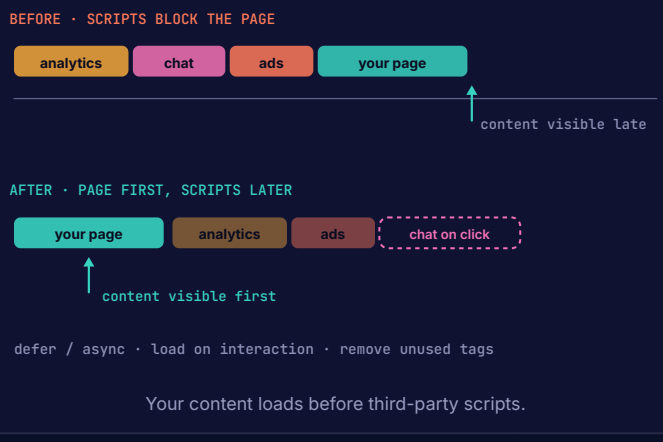
Frontend · IMPACT High · EFFORT Low

WHAT IT IS

Delay analytics, chat widgets, ads, and other third-party scripts until after your page has loaded.

WHY IT MATTERS

Third-party scripts are often the biggest cause of slow loads and laggy taps, and you don't control their size.



Your content loads before third-party scripts.

HOW TO DO IT

- Add `defer` or `async` to script tags, or use your framework's script component with a lazy loading strategy.
- Load non-essential widgets (chat, video embeds) only when the user interacts, using a lightweight placeholder until then.
- Remove any trackers or tools you no longer use.

WATCH OUT

`defer` scripts execute in order after HTML parsing finishes. `async` scripts execute as soon as they're ready, in any order, and can interrupt parsing. Either way, a script still uses the main thread and can change the page when it runs.

HOW TO VERIFY

Run a performance audit before and after: blocking time and main-thread work from third parties should drop.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Optimize third-party scripts.

1. List third-party scripts and embeds in scope (analytics, tag managers, chat, ads, video, maps, social widgets, A/B testing), and flag unused ones for removal.
2. Load the rest with defer (executes in order after HTML parsing) or, for independent scripts, async (executes when ready and can interrupt parsing), or my framework's lazy script strategy.
3. Replace heavy embeds (chat, video, maps) with a lightweight placeholder that loads the real thing on click or when scrolled into view.
4. Keep consent and security scripts working, and note any change in what analytics records.

Report each script and what you changed, with blocking time before and after from a lab audit.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

22 / 31

Split JavaScript by page.

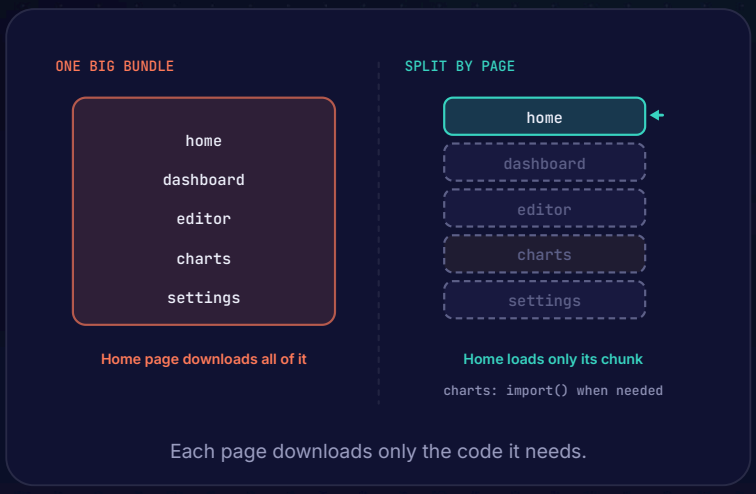
Frontend · IMPACT High · EFFORT Medium

WHAT IT IS

Also called code splitting: each page downloads only the JavaScript it needs, instead of one giant bundle for the whole app.

WHY IT MATTERS

Your home page shouldn't have to download the code for your dashboard, editor, and settings pages.



Each page downloads only the code it needs.

HOW TO DO IT

- Use route-based splitting (most modern frameworks do this automatically per page).
- Load heavy components (charts, rich text editors, maps, PDF viewers) with dynamic `import()` only when needed.
- Keep heavy libraries out of shared layouts that every page loads.

WATCH OUT

Splitting into too many tiny chunks can add extra requests. Focus on large, rarely used components.

HOW TO VERIFY

Use a bundle analyzer: each page's initial JavaScript should be smaller, and heavy libraries should sit in separate chunks.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Reduce how much JavaScript each page loads.

1. Analyze my production bundle: the largest chunks and libraries, and which pages load them.
2. Confirm routes are code-split (many frameworks already do this).
3. Lazy-load heavy components in scope (charts, editors, maps, PDF viewers, modals, admin-only
screens) with dynamic import() and a loading placeholder.
4. Move heavy imports out of shared layouts and root components. Don't split tiny components.

Show the initial JS size per page before and after.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid
services, deleting data and production migrations each need my separate yes.
```

23 / 31

Virtualize long lists.

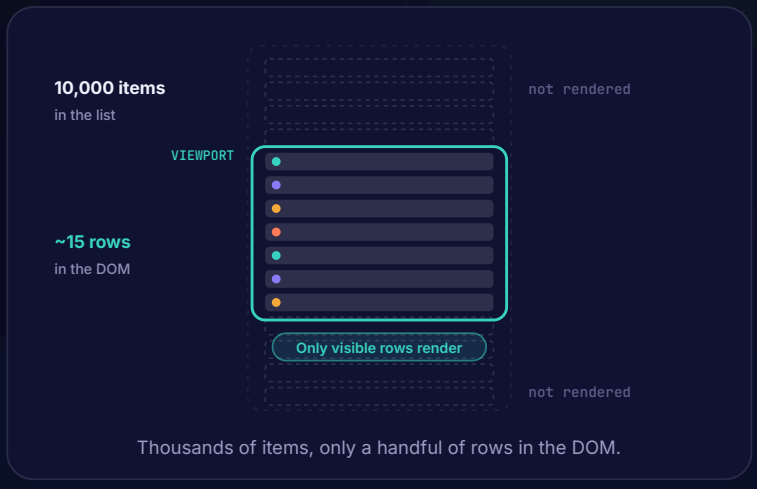
Frontend · IMPACT Medium · EFFORT Medium

WHAT IT IS

Only render the rows that are visible on screen (plus a few extra), even when a list or table has thousands of items.

WHY IT MATTERS

Rendering thousands of rows at once makes scrolling laggy and can freeze the page on slower phones.



Thousands of items, only a handful of rows in the DOM.

HOW TO DO IT

- Use a list-virtualization library or your UI kit's virtualized list/table component.
- Apply it to lists with hundreds of items or more.
- Combine it with pagination or infinite loading so you don't fetch everything either.

WATCH OUT

Browser find-in-page (Ctrl/Cmd+F) can't find rows that aren't rendered, and variable-height rows need extra care. Keep keyboard navigation and accessibility working.

HOW TO VERIFY

Inspect the DOM while scrolling: only a small, roughly constant number of rows should exist at any time.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Virtualize long lists and tables.

1. Find lists, tables, grids, feeds or dropdowns in scope that can hold hundreds of items or more.
Leave short lists alone.
2. Render only visible rows plus a small overscan, using my UI kit's virtualized component or a
well-maintained library that fits my stack.
3. Handle variable row heights if needed, and keep scroll position, keyboard navigation, selection
and accessibility working.
4. Combine with pagination or infinite loading where data comes from the server.

Note that browser find-in-page can't see rows that aren't rendered. List every list you virtualized.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

24 / 31

Select only the columns you need.

Database IMPACT Medium EFFORT Low

WHAT IT IS

Query only the fields a page actually uses, instead of `SELECT *` or fetching whole objects.

WHY IT MATTERS

Less data is read from disk, sent over the network, and serialized, especially when tables have large text or JSON columns.



`SELECT *` reads everything, including big and sensitive columns

`SELECT id, name, email` only what the page shows

Fetch the fields you show, nothing more.

HOW TO DO IT

- Replace `SELECT *` with explicit column lists.
- Use your ORM's select option to pick fields.
- Avoid loading large columns (body text, JSON blobs, files) on list pages.

WATCH OUT

This also improves security: it stops fields like password hashes or internal notes from accidentally reaching the frontend.

HOW TO VERIFY

Compare response size and query time for your main list endpoints before and after.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Make my database queries fetch only the columns they need.

1. Find SELECT * and ORM queries in scope that load full records.
2. Check which fields the calling code and API response actually use, and select only those.
3. Start with large columns (long text, JSON, binary data) on list pages.
4. Make sure sensitive fields (password hashes, tokens, internal notes) are never selected for responses sent to the client.
5. Keep API responses identical for every field the frontend uses.

List each query you changed and the fields removed.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

25 / 31

Use database connection pooling.

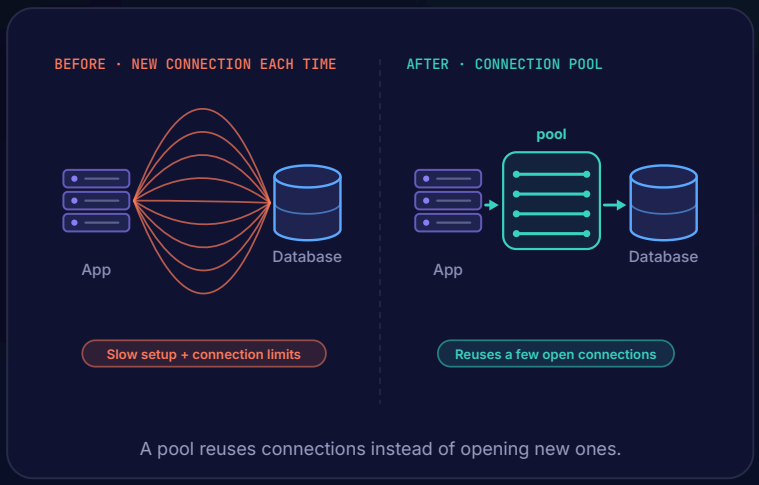
Database · IMPACT High · EFFORT Low

WHAT IT IS

Reuse a small set of open database connections instead of opening a brand-new connection for every request.

WHY IT MATTERS

Opening a connection takes time, and too many connections can crash your database or hit its connection limit.



A pool reuses connections instead of opening new ones.

HOW TO DO IT

- Create one shared database client per server process instead of one per request.
- On serverless platforms, use a connection pooler (many database providers offer a pooled connection URL).
- Set a sensible pool size that stays under your database's connection limit.

WATCH OUT

Some poolers run in transaction mode, which may not support prepared statements or session features. Check your database provider's docs for the right settings.

HOW TO VERIFY

Watch your database's active connection count under load: it should stay stable instead of climbing.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Set up database connection pooling.

1. Find where my app creates database connections or clients, and check whether a new one is created per request or function call.
2. Refactor to one shared client/pool per process, reused across requests (including during development hot reloads).
3. On serverless, use my provider's pooled connection URL or a pooler, and note any settings it requires (e.g., for prepared statements in transaction mode).
4. Set a pool size safely under my database's connection limit, and read settings from environment variables.

Changing production connection settings needs a rollout and rollback plan. Explain how to monitor active connections.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

26 / 31

Prevent unnecessary re-renders.

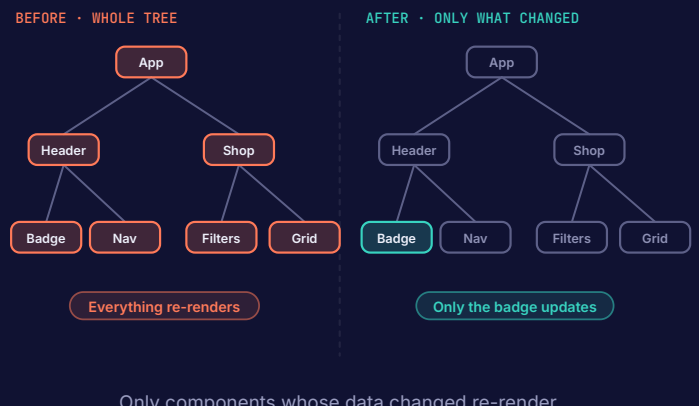
Frontend · IMPACT Medium · EFFORT Medium

WHAT IT IS

Stop components from re-rendering when nothing they display has changed.

WHY IT MATTERS

Unnecessary re-renders make typing, scrolling, and clicking feel laggy, especially on slower phones.



Only components whose data changed re-render.

HOW TO DO IT

- Use your framework's profiler to find components that re-render too often.
- Keep state close to where it's used, and split large contexts that change often.
- Memoize genuinely expensive components and calculations, and keep callback and object references stable.

WATCH OUT

Don't memoize everything. It adds complexity and can be slower. Fix the components the profiler flags first.

HOW TO VERIFY

Record an interaction in the profiler: only the components that actually changed should re-render.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Find and fix unnecessary re-renders.

1. Profile components in scope, or look for common causes: state stored too high up, large
contexts that change often, new objects/arrays/functions every render, expensive calculations
repeated each render, unstable list keys.
2. Fix them by moving state closer to where it's used, splitting contexts, memoizing genuinely
expensive components and calculations, and keeping references stable. Don't wrap everything.
3. Keep behaviour identical.

Tell me how to confirm the improvement with my framework's profiler, and list the components you
changed.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

27 / 31

Enable browser caching for static files.

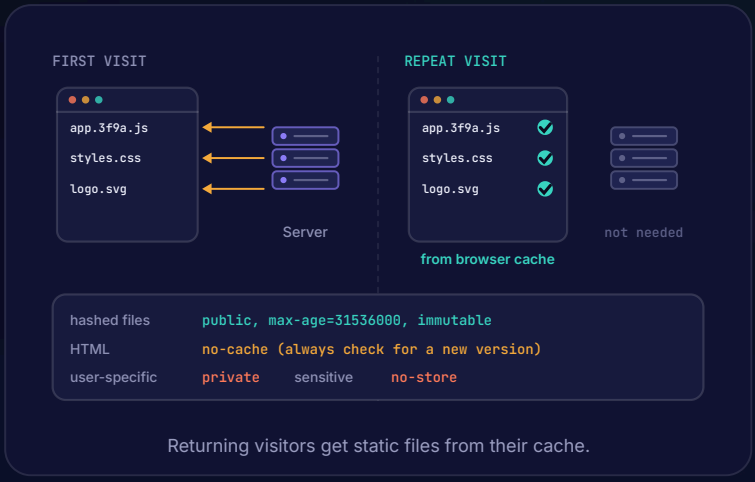
Network · IMPACT Medium · EFFORT Low

WHAT IT IS

Tell browsers to keep copies of static files, so returning visitors don't download them again.

WHY IT MATTERS

Repeat visits load almost instantly because most files come straight from the browser's cache.



Returning visitors get static files from their cache.

HOW TO DO IT

- For versioned files with a hash in the name (like `app.3f9a.js`), use `Cache-Control: public, max-age=31536000, immutable`.
- For HTML, use `no-cache` so the browser always checks for a new version.
- Use `private` for user-specific responses browsers may keep but CDNs must not, and `no-store` for sensitive data that shouldn't be stored at all.

WATCH OUT

Only long-cache files whose names change when their content changes. Otherwise users may be stuck with old versions of your app.

HOW TO VERIFY

Reload the page and check the Network tab: static files should show as served from memory or disk cache.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Set correct browser caching headers.

1. Check the current Cache-Control headers for HTML, JS, CSS, images, fonts and API responses in
scope.
2. Versioned/hashed static files: Cache-Control: public, max-age=31536000, immutable.
3. HTML pages: no-cache, so browsers check for a new version each time.
4. User-specific responses: private (the browser may keep them; CDNs and shared caches must not).
Sensitive responses such as account, payment or personal-data pages: no-store (not stored anywhere).
5. Make sure no unversioned file gets a long cache time.

Header changes affect production behaviour, so note what users might notice and how to roll back. Show
the final header per file type and where it's set.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about
gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs.
Wait for my approval before editing code, config, infrastructure or data; approval covers the whole
plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual
steps for anything you couldn't run. Production deploys, new paid services, deleting data and
production migrations each need my separate yes.
```

28 / 31

Load-balance traffic across multiple app instances.

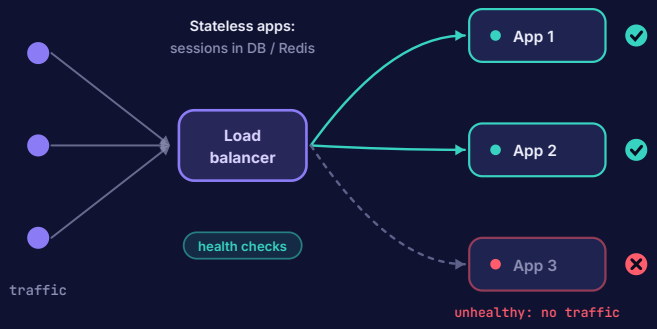
Infra IMPACT Medium EFFORT Medium

WHAT IT IS

Run more than one copy of your app and let a load balancer spread incoming requests between them.

WHY IT MATTERS

No single server gets overwhelmed during traffic spikes, and if one instance crashes the others keep your app online.



Traffic is spread across healthy instances only.

HOW TO DO IT

- Make your app stateless: store sessions, uploads, and caches in a database, object storage, or Redis, not in server memory or local disk.
- Add a health-check endpoint so the load balancer only sends traffic to healthy instances.
- On most hosting platforms, this just means increasing the instance count or enabling autoscaling.

WATCH OUT

Most early-stage apps don't need this yet. If you enable autoscaling, set a maximum instance count so a traffic spike can't run up your bill.

HOW TO VERIFY

Run two or more instances, stop one, and confirm the app keeps working without errors or lost sessions.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Prepare my app to run on multiple instances behind a load balancer.

1. Find state kept in server memory or on local disk (sessions, uploads, in-memory caches, rate-limit
counters, websocket state, scheduled jobs) and propose moving it to shared storage (database, object
storage or Redis).
2. Add a /health endpoint that checks the app and its database connection.
3. Make sure scheduled jobs won't run multiple times when several instances are running.
4. Explain how to enable multiple instances or autoscaling on my host, with a recommended minimum and
maximum instance count.

Because this touches infrastructure, cover: cost per instance, session compatibility for logged-in
users, downtime, and rollback. Label estimates and note what to verify (e.g., provider pricing).

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about
gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs.
Wait for my approval before editing code, config, infrastructure or data; approval covers the whole
plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual
steps for anything you couldn't run. Production deploys, new paid services, deleting data and
production migrations each need my separate yes.
```

29 / 31

Self-host fonts and limit font weights.

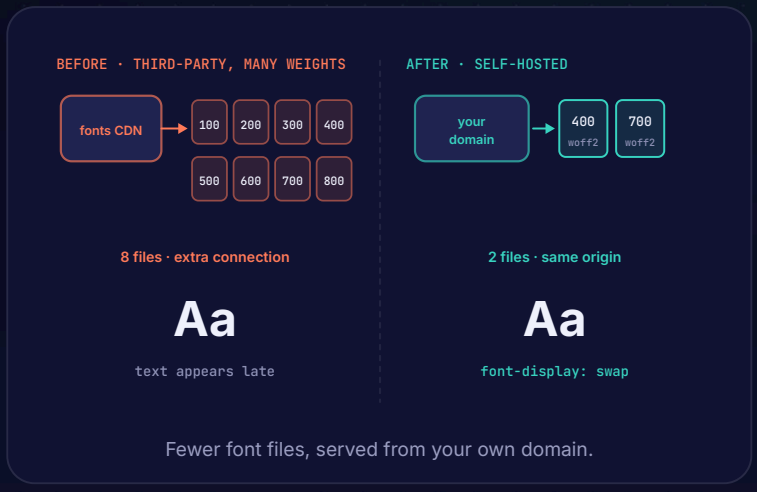
Assets · IMPACT Medium · EFFORT Low

WHAT IT IS

Serve font files from your own domain and only load the weights and styles you actually use.

WHY IT MATTERS

Fonts can block text from appearing. Fewer, smaller font files mean text shows up faster, with no extra connection to a third-party server.



Fewer font files, served from your own domain.

HOW TO DO IT

- Download the fonts in WOFF2 format and serve them yourself, or use your framework's font helper, which often self-hosts automatically.
- Limit to 2–3 weights, or use one variable font file.
- Use `font-display: swap`, and preload only the main font file.

WATCH OUT

Check the font's license allows self-hosting. Preloading too many fonts slows the page down instead of speeding it up.

HOW TO VERIFY

In the Network tab, fonts should load from your own domain, and only a few font files should download.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Optimize the fonts in my app.

1. Find every font my app loads and the weights/styles actually used in CSS and components.
2. Self-host them as WOFF2 or use my framework's font optimization, removing third-party font
requests. Check the license allows self-hosting.
3. Remove unused weights and styles, keeping 2-3 weights or a single variable font.
4. Use font-display: swap, preload only the primary font file, and subset where supported.
5. Add a matching fallback font stack to minimize layout shift.

Show which font files are loaded before and after.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

30 / 31

Track real-user speed with a performance monitoring tool.

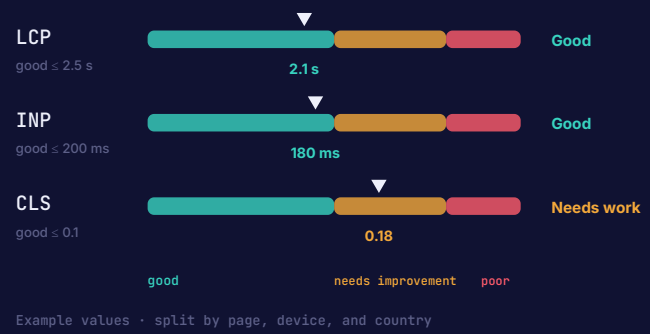
Monitoring IMPACT High EFFORT Low

WHAT IT IS

Measure how fast your app actually is for real users on their real devices and networks (called Real User Monitoring), not just on your fast laptop.

WHY IT MATTERS

Your users' phones and connections are usually much slower than yours. Real data shows which pages, devices, and countries need work.



Track Core Web Vitals from real visits.

HOW TO DO IT

- Track Core Web Vitals: LCP (loading), INP (responsiveness), and CLS (visual stability).
- Aim for good scores at the 75th percentile: LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1.
- Break results down by page, device type, and country.

WATCH OUT

Respect privacy laws and consent requirements, and don't collect personal data in performance events.

HOW TO VERIFY

After a few days of traffic, you should see Core Web Vitals per page in your monitoring dashboard.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Set up real-user performance monitoring.

1. Check what my host or existing analytics already reports, and prefer that before adding a
service.
2. Collect Core Web Vitals (LCP, INP, CLS) and page load time from real users, tagged by route,
device type and connection type. Add API response times if possible.
3. Respect user consent and keep personal data out of performance events.
4. Set targets using Google's "good" thresholds at the 75th percentile: LCP ≤ 2.5 s, INP ≤ 200 ms,
CLS ≤ 0.1.

If this adds a service, cover cost, data retention and privacy settings. Explain how to read the
dashboard and find my worst pages.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only
about gaps that change the plan. Propose the smallest useful plan with its benefit and any real
trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval
covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real
results, with manual steps for anything you couldn't run. Production deploys, new paid services,
deleting data and production migrations each need my separate yes.
```

31 / 31

Test on slow phones and networks.

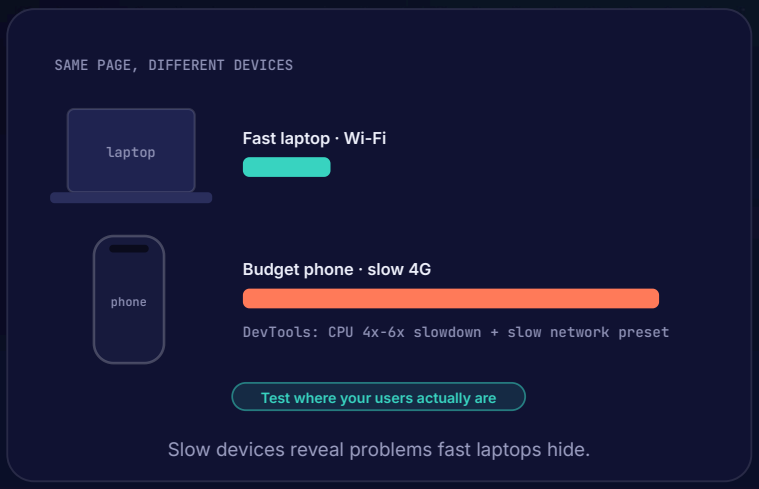
Monitoring · IMPACT High · EFFORT Low

WHAT IT IS

Test your app the way most of the world actually uses it: on mid-range phones over slow mobile connections.

WHY IT MATTERS

An app that feels instant on a new laptop over fast Wi-Fi can feel painfully slow on a budget phone over a weak connection.



Slow devices reveal problems fast laptops hide.

HOW TO DO IT

- Use your browser DevTools to throttle CPU (4x–6x slowdown) and network (slow 4G/3G presets).
- Run a mobile lab audit on your key pages (it covers LCP and CLS; INP needs real interactions).
- Test on at least one real mid-range or budget Android phone.

WATCH OUT

Emulation is only an approximation. Real devices often reveal problems DevTools doesn't.

HOW TO VERIFY

Your key pages should load and respond acceptably with throttling on, and you can set performance budgets to keep them that way.

COPY-PASTE PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

Help me test and improve my app on slow phones and networks.

1. Give me a test plan for my 5 most important pages and flows, using DevTools CPU throttling (4x-6x) and slow network presets.
2. Run, or tell me how to run, a mobile lab audit on those pages and summarize the biggest issues. Lab audits report LCP, CLS and Total Blocking Time; INP needs real interactions or real-user data.
3. Rank the fixes by impact on slow devices: JS size, images, fonts, third-party scripts, long tasks and request waterfalls.
4. Propose performance budgets (e.g., max JS per page, image weight, LCP target) and, if my CI supports it, a check that fails the build when one is exceeded.

Start with the fixes that help slow devices the most.

How to work: inspect first ("already done" or "no change needed" are fine answers) and ask only about gaps that change the plan. Propose the smallest useful plan with its benefit and any real trade-offs. Wait for my approval before editing code, config, infrastructure or data; approval covers the whole plan unless scope, cost or risk changes. Then implement, test, and report real results, with manual steps for anything you couldn't run. Production deploys, new paid services, deleting data and production migrations each need my separate yes.
```

PROMPT 32

The final *check.*

When you've worked through the list, run this prompt to measure your results, confirm nothing broke, and catch any security mistakes the changes may have introduced.

FINAL CHECK PROMPT

```
Context: [paste your project context]   Scope: [one page, endpoint, service or "whole app"]

We've finished the performance work. Now verify everything.

1. Re-run the measurements from PERFORMANCE.md the same way (lab audit vs. real-user data, same pages
and settings) and fill in the "After" column.
2. Run the full test suite and build, and click through the main user flows to confirm nothing broke.
3. Double-check for mistakes introduced by the changes: caches keyed per user, no private data cached by
the CDN or browser, no secrets in server-rendered HTML, and no unsafe retries on payments.
4. Summarize the before/after results, what you couldn't measure, and anything still worth improving.

Only verify and report; don't deploy or change anything further without asking me.
```

Targets to aim for

METRIC	WHAT IT MEASURES	GOOD
LCP	Largest Contentful Paint: how fast the main content appears	≤ 2.5 s
INP	Interaction to Next Paint: how fast the page responds to taps and clicks	≤ 200 ms
CLS	Cumulative Layout Shift: how much the page jumps around	≤ 0.1
TTFB	Time to First Byte: how fast the server starts responding	≤ 0.8 s
Lighthouse	Lab score from a mobile audit (measures LCP and CLS, not INP)	90+

Web Vitals thresholds are Google's "good" ratings for real-user data at the 75th percentile. INP needs real interactions; in lab tests, Total Blocking Time is a rough guide.

> **Speed and safety go together**
> Several fixes in this toolkit (caching, CDNs, server rendering, retries) can leak data or cause duplicate charges if they're done carelessly. Each card's "Watch out" box flags those risks. Read them before you ship.

PLEASE READ

Copyright & license.

© 2026 BreakingBreach. All rights reserved. This toolkit, including its text, checklist, prompts, diagrams, illustrations, design, and layout, is protected by copyright and other intellectual property laws.

What you may do

You are granted a limited, personal, non-exclusive, non-transferable, revocable license to keep one copy of this toolkit for your own learning, and to copy individual prompts into your own AI tools to work on your own apps, including commercial apps.

What you may not do

Unless you have written permission, you may not:

- Share, send, forward, or give this toolkit, or any part of it, to anyone else.
- Upload, post, or publish it anywhere, including websites, social media, groups, communities, messaging apps, file-sharing services, or cloud links.
- Reproduce, copy, print, or republish all or part of it, except as allowed above.
- Sell, resell, rent, license, or bundle it, or use it in any product, course, membership, newsletter, or giveaway.
- Modify, translate, adapt, or repackage it, or present the checklist, prompts, or diagrams as your own work.
- Remove or change any copyright notice, attribution, or this page.
- Use it to train, fine-tune, or evaluate AI models, or include it in any dataset, or scrape or extract its content in bulk.

If someone else wants it

Please don't send them your copy. Point them to @breakingbreach so they can get their own.

Ending of this license

This license ends automatically if you break any of these terms. If it ends, you must stop using the toolkit and delete all copies you have. Unauthorized use may result in takedown requests or other legal action.

Your other rights

Nothing in these terms limits any rights you have under applicable law that cannot be waived.

PLEASE READ

Disclaimer.

Educational purposes only

This toolkit is provided for general educational and informational purposes only. It is not professional, technical, security, legal, or financial advice, and using it does not create any professional or client relationship.

No warranty

All content is provided "as is" and "as available", without warranties of any kind, express or implied, including accuracy, completeness, reliability, merchantability, fitness for a particular purpose, or non-infringement.

Accuracy and changes

The content reflects common practices as of November 2026. Frameworks, platforms, and best practices change often, so some information may become outdated, incomplete, or not apply to your project. This toolkit may be updated or withdrawn at any time without notice.

Do your own research

This toolkit is a starting point, not a complete or authoritative source. Before using any information, technique, or prompt, independently research and verify it against official documentation, your service providers' guidance, and qualified professionals, and confirm it is suitable, safe, and lawful for your specific situation.

Examples and results

Diagrams are simplified illustrations, and the numbers, timings, and sizes in them are examples, not measurements. Actual results depend on your app, stack, hosting, data, and users. No particular result is promised or guaranteed.

You are responsible for your app

You are solely responsible for any changes you make to your software, systems, data, and infrastructure. Always back up your data, use version control, test changes in a non-production environment first, and have code reviewed before deploying it.

AI-generated code

The prompts are designed for AI coding tools. AI tools can make mistakes, misunderstand instructions, introduce bugs or security vulnerabilities, delete or alter data, or produce code that does not work as intended. Review, understand, and test every change before you use it.

Not a security or compliance review

Following this toolkit does not make your app secure, and it does not make it compliant with any law, regulation, or standard, including privacy and data-protection laws. Get qualified professional advice where you need it.

Costs

Some suggestions involve third-party services, infrastructure, or AI tools that may charge fees. You are solely responsible for any costs, usage charges, or bills you incur, including unexpected ones.

Assumption of risk

You use this toolkit entirely at your own risk. You accept full responsibility for any consequences of acting on it, including security incidents, data loss or exposure, downtime, performance problems, unexpected costs, legal or compliance issues, and any other loss or harm, whether or not foreseeable.

Limitation of liability

To the maximum extent permitted by law, the creator of this toolkit shall not be liable for any direct, indirect, incidental, consequential, special, or punitive damages, or for any loss of data, revenue, profits, or business, downtime, security incidents, service costs, or any other losses or harm arising from or related to the use of, or inability to use, this toolkit, even if advised of the possibility of such damages.

Indemnification

You agree to indemnify and hold harmless the creator of this toolkit from any claims, damages, losses, or expenses arising from your use or misuse of this toolkit or your breach of these terms.

Third-party services and trademarks

Mentions of any technology, product, standard, or service are for information only and do not imply endorsement, sponsorship, or affiliation. Third-party services are subject to their own terms and policies. All trademarks belong to their respective owners.

Privacy

This PDF contains no forms, scripts, or tracking, and collects no data about you.

Severability and acceptance

If any part of these terms is found unenforceable, the rest remains in effect. By using this toolkit, you agree to this disclaimer and the license terms on the previous page.



Now go make it *fast.*

Found this useful? There's more where it came from.

Follow @breakingbreach

© 2026 BreakingBreach. All rights reserved.
Personal use only. Do not share, copy, or redistribute.